Tugas Praktikum Ensemble Method dan Class Imbalance

Malfino Muhammad Willianz - 5054251028

Dept. Teknik Informatika, Prodi Rekayasa Kecerdasan Artifisial

1. Deskripsi Dataset

Dataset yang digunakan pada praktikum ini adalah *Credit Card Fraud Detection* yang diperoleh dari Kaggle. Dataset berisi catatan transaksi kartu kredit selama dua hari, dengan tujuan membedakan transaksi normal dan transaksi penipuan.

Berkas yang digunakan adalah `creditcard.csv` dengan 284.807 baris dan 31 kolom. Atribut `Class` merupakan label target, di mana nilai `0` menandakan transaksi normal dan nilai `1` menandakan transaksi penipuan. Sebanyak 28 atribut, yaitu `V1` sampai `V28`, merupakan hasil transformasi *Principal Component Analysis* (PCA) dari atribut asli yang dirahasiakan untuk melindungi privasi pemegang kartu. Atribut `Time` mencatat selisih waktu dalam detik dari transaksi pertama, sedangkan `Amount` mencatat nilai transaksi.

Secara umum, atribut pada dataset dapat dikelompokkan menjadi beberapa tipe:

Atribut	Tipe Atribut	Penjelasan
`Time`	Numerik Kontinu	Selisih waktu dalam detik dari transaksi pertama. Tidak bermakna langsung sehingga diturunkan menjadi fitur jam.
`V1–V28`	Numerik Kontinu	Komponen hasil PCA yang telah dinormalisasi dan saling ortogonal.
`Amount`	Numerik Kontinu	Nilai transaksi dalam satuan mata uang, dengan skala jauh lebih besar dibanding `V1–V28`.
`Class`	Nominal Biner	Label target. `0` = transaksi normal, `1` = transaksi penipuan (*fraud*).

Pemeriksaan awal menunjukkan dataset tidak memiliki nilai hilang, sehingga tahapan praproses lebih berfokus pada penanganan skala fitur dan distribusi kelas yang tidak seimbang.

2. Eksplorasi Distribusi Kelas

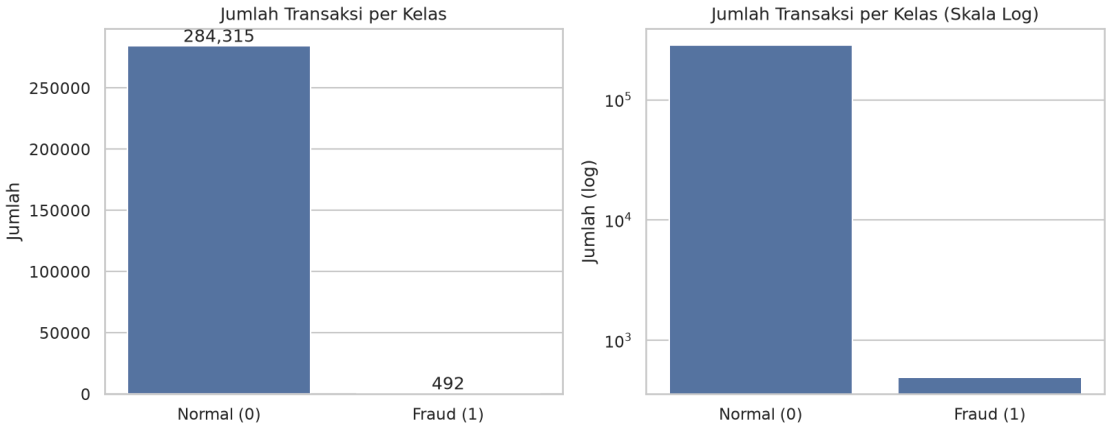
2.1. Ketidakseimbangan Kelas

Langkah pertama adalah mengidentifikasi distribusi kelas pada atribut `Class`. Hasilnya menunjukkan ketidakseimbangan yang sangat ekstrem.

Kelas	Jumlah	Persentase

Normal (0)	284.315	99,8273%
Fraud (1)	492	0,1727%

Rasio antara kelas mayoritas dan kelas minoritas mencapai sekitar **578:1**. Kondisi ini berarti kelas minoritas hampir tidak terwakili dalam dataset.



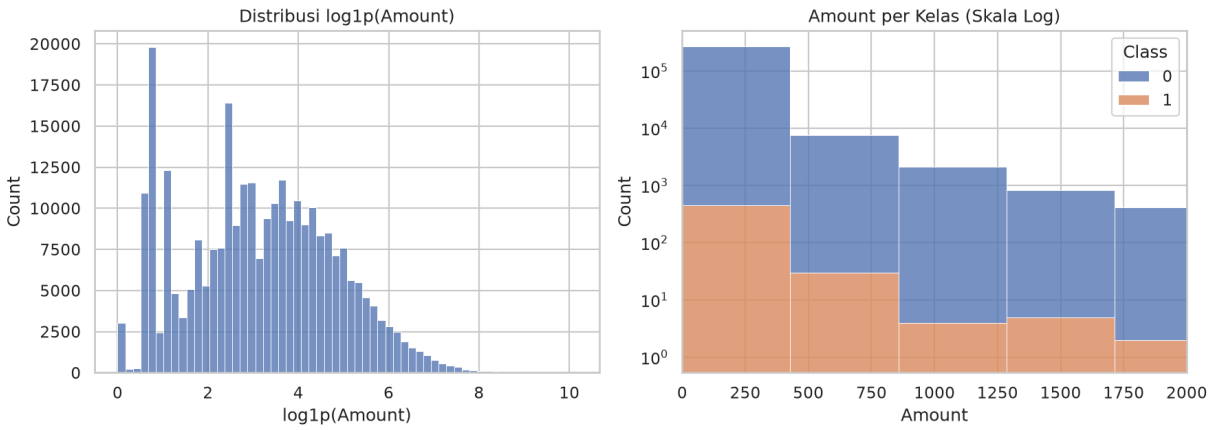
Gambar 1: Distribusi Jumlah Transaksi per Kelas (Skala Linear dan Logaritmik)

Perbedaan jumlah yang sangat besar membuat akurasi menjadi metrik yang menyesatkan. Model yang selalu memprediksi kelas normal akan memperoleh akurasi di atas 99%, tetapi sama sekali tidak mampu mendeteksi penipuan. Oleh karena itu, penanganan *class imbalance* menjadi langkah yang wajib sebelum pemodelan.

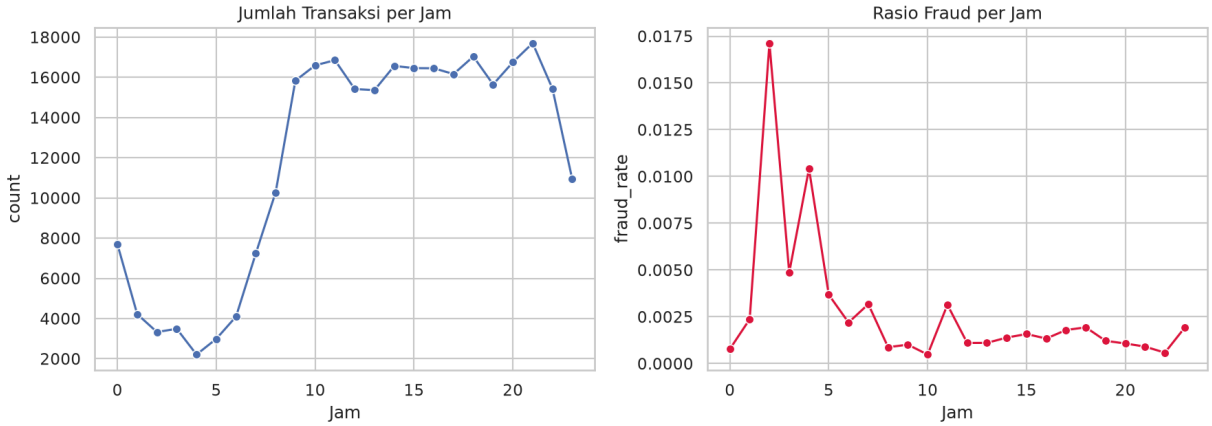
2.2. Statistik Deskriptif dan Pola Fitur

Atribut `Amount` memiliki distribusi yang sangat menceng ke kanan. Nilai rata-ratanya sebesar 88,35 sedangkan median hanya 22,00, yang mengindikasikan keberadaan sejumlah transaksi bernilai sangat besar.

Statistik	**Amount**	**Time (detik)**
Jumlah observasi	284.807	284.807
Rata-rata	88,35	94.813,86
Standar deviasi	250,12	47.488,15
Minimum	0,00	0,00
Median	22,00	84.692,00
Maksimum	25.691,16	172.792,00

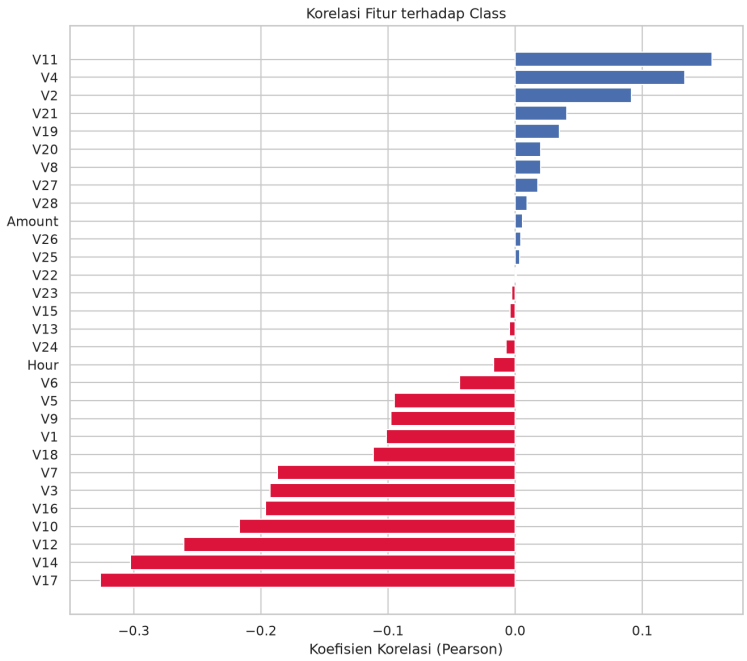


Gambar 2: Distribusi `Amount` dan Perbandingannya Antar Kelas



Gambar 3: Jumlah Transaksi dan Rasio Fraud per Jam

Dari sisi waktu, jumlah transaksi mengikuti pola harian dengan aktivitas terendah pada dini hari. Menariknya, rasio penipuan justru meningkat pada jam-jam dengan volume transaksi rendah, terutama sekitar pukul 02.00 sampai 05.00. Pola ini menunjukkan bahwa `Time` menyimpan informasi yang relevan dan lebih berguna ketika diturunkan menjadi fitur jam.



Gambar 4: Korelasi Setiap Fitur terhadap Kelas Target

Beberapa komponen PCA memiliki korelasi yang cukup kuat terhadap kelas target. `V17`, `V14`, `V12`, dan `V10` berkorelasi negatif, sedangkan `V11`, `V4`, dan `V2` berkorelasi positif. Sebaliknya, `Amount` hanya memiliki korelasi yang sangat lemah, sehingga nilai transaksi saja tidak cukup untuk membedakan penipuan.

3. Praproses Data

3.1. Penskalaan Fitur

Fitur `Amount` distandarisasi menggunakan *z-score* melalui `StandardScaler` agar skalanya sebanding dengan `V1`–`V28` yang telah dinormalisasi oleh PCA. Fitur `Time` diubah menjadi fitur `Hour` karena nilai detik mentah tidak bermakna langsung bagi model. Atribut asli `Time` dan `Amount` kemudian dihapus untuk menghindari redundansi.

Dataset tidak mengandung nilai hilang sehingga tidak diperlukan tahap imputasi.

3.2. Pembagian Data Latih dan Uji

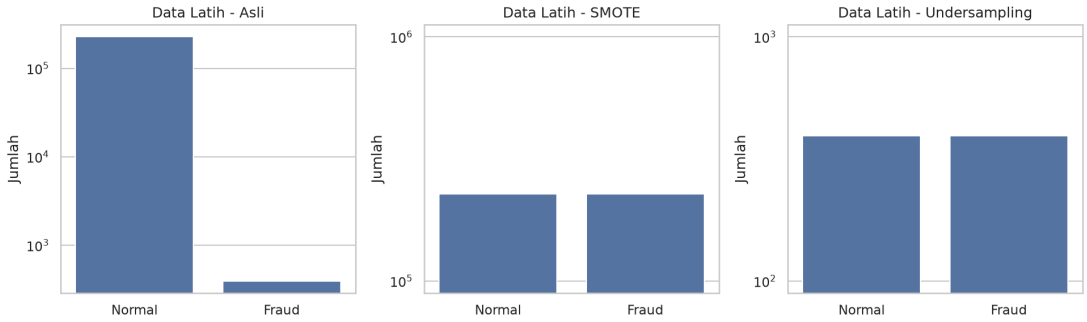
Dataset dibagi menjadi data latih dan data uji dengan perbandingan 80:20 menggunakan parameter `stratify=y` agar proporsi kelas tetap terjaga. Hasilnya, data latih berisi 227.845 baris dengan 394 transaksi penipuan (0,1729%), sedangkan data uji berisi 56.962 baris dengan 98 transaksi penipuan (0,1720%).

4. Penanganan Class Imbalance

Tiga pendekatan penanganan *class imbalance* diterapkan pada data latih:

- **SMOTE** (*oversampling*). Sampel sintetis pada kelas minoritas dibangkitkan melalui interpolasi antar tetangga terdekat. Jumlah kelas fraud pada data latih meningkat dari 394 menjadi 227.451 sehingga menjadi seimbang, tanpa menduplikasi data secara langsung.

- **Undersampling**. Kelas mayoritas dikurangi dengan membuang sebagian transaksi normal hingga jumlahnya seimbang dengan kelas fraud. Data latih menyusut drastis dari 227.845 menjadi hanya 788 baris.
- **Cost-sensitive learning** (*class weighting*). Data tidak diubah, tetapi model diberi penalti lebih besar pada kesalahan klasifikasi kelas minoritas melalui parameter `class_weight`.



Gambar 5: Distribusi Kelas pada Data Latih Sebelum dan Sesudah Resampling

Resampling hanya diterapkan pada data latih. Data uji tetap menggunakan distribusi aslinya, yaitu dengan rasio penipuan 0,17%, agar evaluasi mencerminkan performa model pada kondisi nyata.

5. Membangun Model Klasifikasi

Seluruh model yang dibandingkan merupakan *ensemble* pada level model, yaitu model yang menggabungkan banyak model dasar di dalam proses pelatihannya. Dua keluarga utama digunakan sesuai ketentuan praktikum, kemudian ditambahkan tiga implementasi *gradient boosting* modern sebagai pembanding:

- **Random Forest**, yaitu metode *bagging* yang membangun banyak pohon keputusan pada sampel *bootstrap* dan menggabungkan hasilnya melalui *voting*. Konfigurasi yang digunakan adalah 200 pohon.
- **Gradient Boosting**, yaitu metode *boosting* yang membangun pohon secara berurutan untuk memperbaiki kesalahan model sebelumnya. Implementasi yang digunakan adalah `HistGradientBoostingClassifier` dengan 300 iterasi dan *learning rate* 0,1.
- **CatBoost, XGBoost, dan LightGBM**, yaitu tiga implementasi *gradient boosting* modern dengan mekanisme penanganan *class imbalance* bawaan.

Random Forest dan Gradient Boosting masing-masing diuji pada empat konfigurasi, yaitu baseline, *class weighting*, SMOTE, dan undersampling. Ketiga *boosting* modern ditambahkan pada konfigurasi terbaiknya masing-masing, sehingga terdapat sebelas model yang dibandingkan pada data uji yang sama.

Penanganan *imbalance* pada ketiga *boosting* modern dilakukan melalui pembobotan gradien pada fungsi objektif:

- **XGBoost** menggunakan `scale_pos_weight` sebesar 577,29, yang mengalikan gradien kelas minoritas sehingga kesalahan pada kelas fraud dianggap lebih mahal.

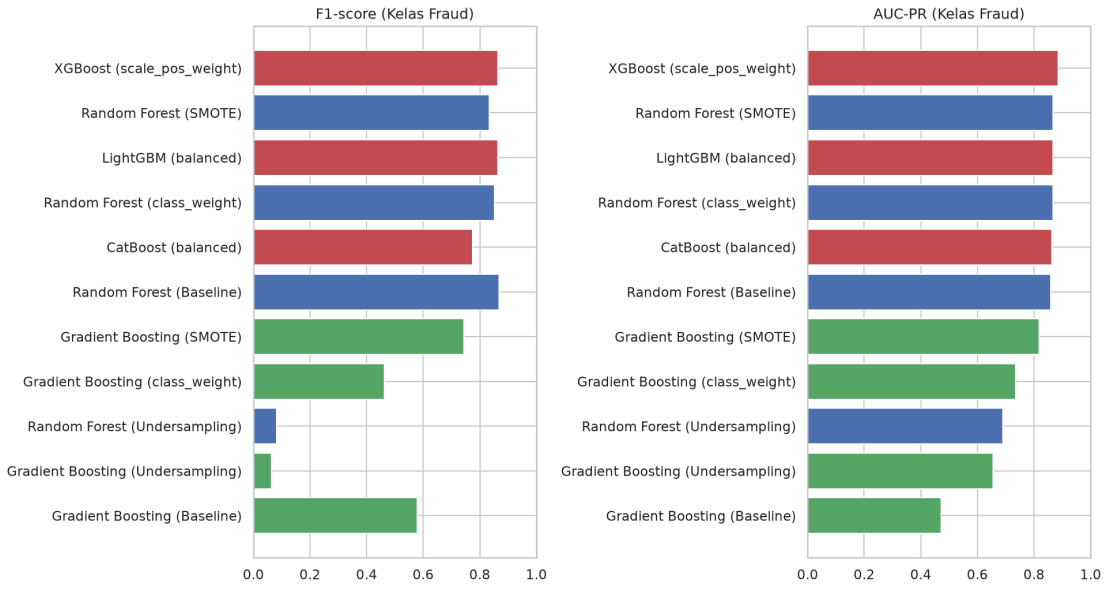
- **LightGBM** menggunakan `class_weight="balanced"`, yang memberikan bobot berbanding terbalik dengan frekuensi kelas.
- **CatBoost** menggunakan `auto_class_weights="Balanced"`, yang menyeimbangkan bobot kelas secara otomatis selama pelatihan.

6. Evaluasi Model

6.1. Perbandingan Metrik

Evaluasi difokuskan pada kelas minoritas (fraud) menggunakan *precision*, *recall*, F1-score, dan AUC-ROC. Karena dataset sangat tidak seimbang, ditambahkan pula metrik AUC-PR (*average precision*) yang lebih sensitif terhadap kelas minoritas.

Model	Prec.	Recall	F1	ROC	AUC-PR	Waktu
RF Baseline	0,9405	0,8061	0,8681	0,9619	0,8584	73,8 s
RF class_weight	0,9176	0,7959	0,8525	0,9608	0,8663	29,3 s
RF SMOTE	0,8511	0,8163	0,8333	0,9750	0,8679	133,9 s
RF Undersampling	0,0430	0,9184	0,0822	0,9780	0,6909	0,5 s
GB Baseline	0,5250	0,6429	0,5780	0,7258	0,4721	0,7 s
GB class_weight	0,3110	0,8980	0,4619	0,9497	0,7343	0,9 s
GB SMOTE	0,6466	0,8776	0,7446	0,9572	0,8172	4,3 s
GB Undersampling	0,0331	0,9184	0,0638	0,9750	0,6547	0,6 s
CatBoost	0,7059	0,8571	0,7742	0,9683	0,8626	7,2 s
XGBoost	0,8830	0,8469	0,8646	0,9686	**0,8847**	6,3 s
LightGBM	**0,9195**	0,8163	**0,8649**	0,9604	0,8677	5,5 s



Gambar 6: Perbandingan F1-score dan AUC-PR Antar Model

Pada keluarga *ensemble* inti, Random Forest bersifat jauh lebih tahan terhadap ketidakseimbangan kelas dibanding Gradient Boosting. Bahkan pada konfigurasi baseline,

Random Forest sudah mencapai *recall* 0,8061 dan AUC-PR 0,8584, sedangkan Gradient Boosting baseline hanya mencapai *recall* 0,6429 dengan AUC-PR 0,4721. Hal ini karena setiap pohon pada Random Forest dibangun dari sampel *bootstrap* dengan pemilihan *split* pada subset fitur acak, sehingga kelas minoritas tetap sering terwakili pada simpul-simpul pohon. Sebaliknya, pada Gradient Boosting setiap pohon baru berfokus memperbaiki sisa kesalahan yang didominasi kelas mayoritas.

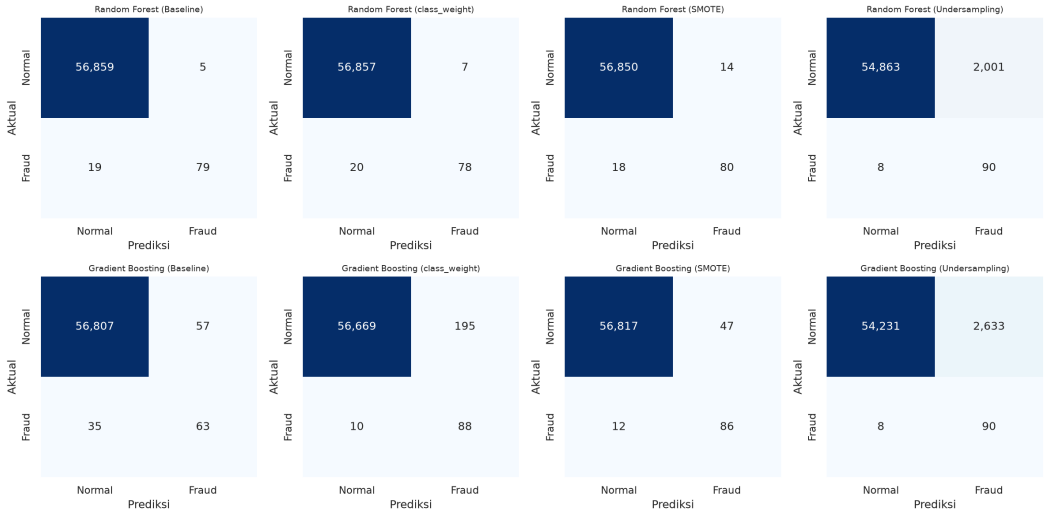
Setelah penanganan *imbalance*, performa Gradient Boosting melonjak tajam: SMOTE menaikkan *recall* menjadi 0,8776 dan AUC-PR menjadi 0,8172. Hal ini menunjukkan bahwa Gradient Boosting sangat bergantung pada penanganan *imbalance*, sedangkan Random Forest jauh lebih mandiri.

Ketiga *boosting* modern tampil sangat kuat dan konsisten. XGBoost dengan `scale_pos_weight` menjadi model tunggal terbaik secara AUC-PR, yaitu 0,8847, disusul LightGBM (0,8677) dan CatBoost (0,8626). Ketiganya tidak memerlukan data sintetis karena penanganan *imbalance* dilakukan melalui bobot gradien internal, sehingga waktu pelatihannya hanya 5–7 detik, jauh lebih singkat dibanding Random Forest dengan SMOTE yang memerlukan 134 detik.

Undersampling menghasilkan *recall* tertinggi pada kedua keluarga *ensemble* inti, yaitu 0,9184, tetapi *precision*-nya anjlok hingga 0,0330–0,0430. Hal ini wajar karena data latih menyusut drastis, sehingga model kehilangan banyak informasi kelas mayoritas dan menjadi terlalu agresif menandai fraud.

AUC-ROC seluruh model tampak tinggi (0,94–0,98) dan sulit dibedakan, sedangkan AUC-PR menyebar cukup lebar (0,47–0,88). Perbedaan ini menegaskan bahwa AUC-PR jauh lebih mampu membedakan kualitas model pada dataset yang sangat tidak seimbang.

6.2. Confusion Matrix



Gambar 7: Confusion Matrix Random Forest dan Gradient Boosting pada Data Uji



Gambar 8: Confusion Matrix CatBoost, XGBoost, dan LightGBM pada Data Uji

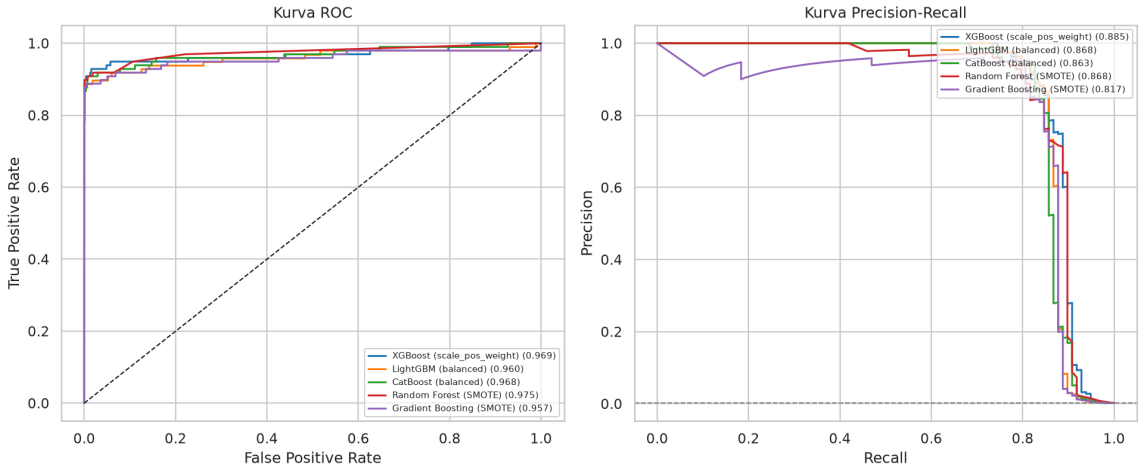
Confusion matrix memperjelas *trade-off* antar pendekatan. Pada Random Forest baseline, terdapat 19 *false negative* (penipuan yang lolos) dan hanya 5 *false positive*. Setelah SMOTE, *false negative* turun menjadi 18 dengan *false positive* 14.

Perbedaan yang jauh lebih mencolok terlihat pada Gradient Boosting. Baseline menghasilkan 35 *false negative*, lalu turun menjadi 10 dengan *class weighting* dan 12 dengan SMOTE. Penurunan ini menunjukkan bahwa penanganan *imbalance* pada Gradient Boosting tidak hanya menaikkan metrik secara agregat, tetapi benar-benar mengurangi jumlah penipuan yang lolos.

Undersampling pada kedua keluarga menekan *false negative* paling rendah, yaitu 8, tetapi konsekuensinya *false positive* meledak hingga 2.001 sampai 2.633 transaksi normal yang salah ditandai. Hal ini membuat pendekatan tersebut kurang praktis meskipun *recall*-nya tinggi.

Ketiga *boosting* modern menghasilkan confusion matrix yang seimbang. LightGBM hanya melewatkan 18 penipuan dengan 7 *false positive*, sedangkan XGBoost dan CatBoost masing-masing melewatkan 15 dan 14 penipuan dengan *false positive* 11 dan 35. Pada kasus deteksi penipuan, *false negative* jauh lebih merugikan karena penipuan yang tidak terdeteksi menimbulkan kerugian finansial langsung, sedangkan *false positive* hanya menambah beban verifikasi. Kombinasi yang dicapai ketiga model ini karena itu merupakan hasil yang paling diinginkan.

6.3. Kurva ROC dan Precision-Recall



Gambar 9: Kurva ROC dan Kurva Precision-Recall Model Terpilih

Kurva ROC menunjukkan hampir seluruh model memiliki AUC-ROC di atas 0,94, sehingga kurva tampak berdekatan dan sulit dibedakan. Hal ini merupakan konsekuensi dari dominasi kelas mayoritas, di mana *true negative* yang sangat banyak membuat *false positive rate* tetap rendah sehingga kurva ROC cenderung bergerak mendekati sudut kiri atas.

Kurva Precision-Recall jauh lebih informatif untuk dataset tidak seimbang. Garis putus-putus merepresentasikan performa model acak, yaitu sebesar prevalensi kelas fraud (0,0017). Di sini pemisahan antar model menjadi jelas: XGBoost dan LightGBM berada paling atas, diikuti CatBoost dan Random Forest, sedangkan Gradient Boosting baseline berada jauh di bawah.

Bentuk kurva ketiga *boosting* modern juga lebih stabil pada area *recall* tinggi dibanding Random Forest. Hal ini menunjukkan bahwa selain lebih cepat dilatih, ketiganya menghasilkan peringkat probabilitas yang lebih tajam pada transaksi yang paling mencurigakan.

6.4. Mengapa Accuracy Tidak Cukup

Untuk menunjukkan mengapa akurasi tidak memadai, dibandingkan performa model yang selalu memprediksi kelas normal dengan model XGBoost yang terbaik.

Model	**Accuracy**	**Recall (Fraud)**	**F1 (Fraud)**
Prediksi selalu Normal	0,9983	0,0000	0,0000
XGBoost	0,9995	0,8469	0,8646

Model yang selalu memprediksi kelas normal memperoleh akurasi 0,9983. Angka ini terlihat sangat baik, tetapi model tersebut sama sekali tidak berguna karena tidak mampu mendeteksi satu pun transaksi penipuan (*recall* = 0).

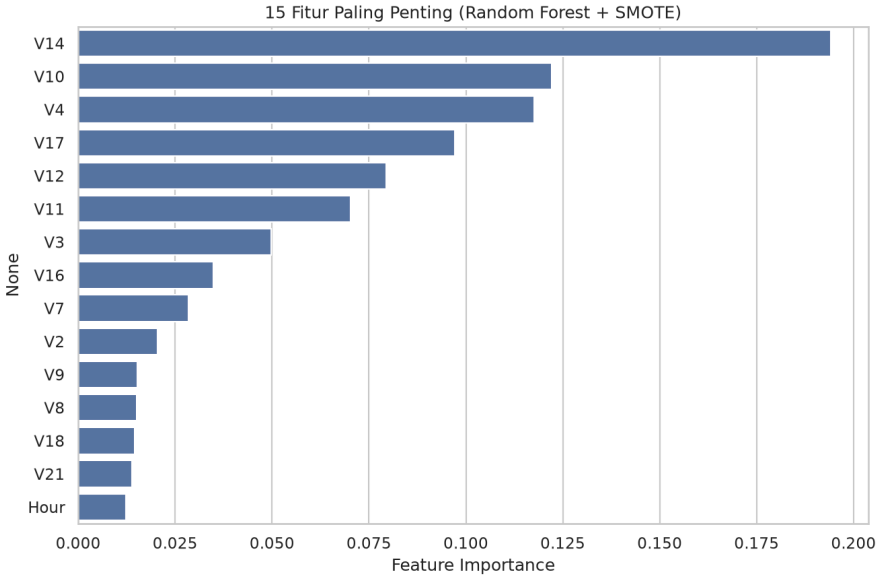
Akurasi mengukur proporsi keseluruhan prediksi yang benar, sehingga setiap transaksi normal yang diprediksi benar menyumbang nilai akurasi yang sama besarnya dengan

transaksi penipuan. Ketika kelas mayoritas mendominasi 99,8% data, akurasi menjadi tidak sensitif terhadap kesalahan pada kelas minoritas. Padahal pada kasus deteksi penipuan, kelas minoritas justru merupakan kelas yang paling penting untuk dikenali.

Oleh karena itu, metrik yang lebih sesuai adalah *precision*, *recall*, F1-score, dan AUC-PR yang berfokus pada performa pada kelas minoritas. AUC-ROC tetap dapat dilaporkan, tetapi harus dibaca dengan hati-hati karena dapat memberikan kesan performa yang terlalu optimistis pada data yang sangat tidak seimbang.

7. Analisis Fitur dan Threshold

7.1. Feature Importance

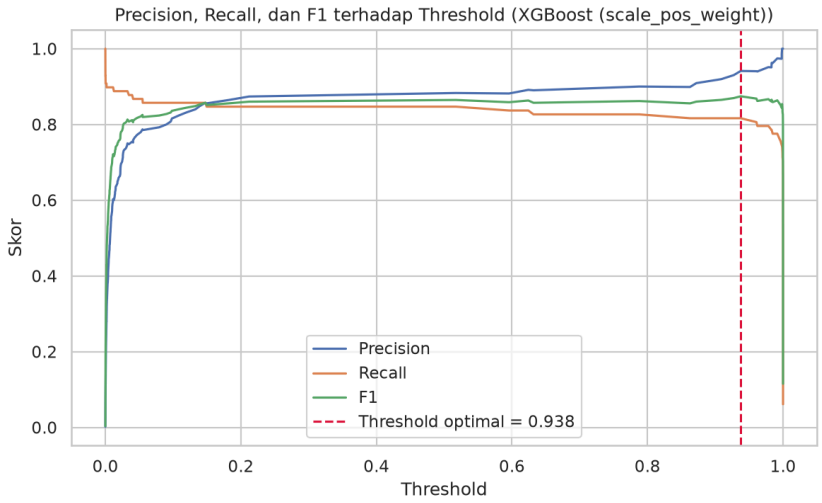


Gambar 10: 15 Fitur Paling Penting pada Random Forest dengan SMOTE

Fitur yang paling berpengaruh pada model didominasi oleh komponen PCA tertentu. `V14` menjadi fitur terpenting dengan kontribusi mendekati 0,20, diikuti `V10`, `V4`, `V17`, dan `V12`. Urutan ini konsisten dengan hasil analisis korelasi sebelumnya, di mana `V14` dan `V17` berkorelasi negatif kuat terhadap kelas fraud, sedangkan `V4` dan `V11` berkorelasi positif. Fitur turunan `Hour` juga masuk ke dalam 15 fitur teratas, sedangkan `Amount_scaled` tidak muncul sehingga kontribusinya relatif kecil.

Temuan ini menegaskan bahwa pola penipuan lebih tercermin pada kombinasi pola tersembunyi dalam `V1–V28` dibandingkan pada nilai transaksi itu sendiri.

7.2. Threshold Tuning



Gambar 11: Precision, Recall, dan F1 terhadap Threshold pada Model Terbaik

Ambang batas keputusan secara langsung mempengaruhi keseimbangan antara *precision* dan *recall*. Pada model terbaik, yaitu XGBoost dengan `scale_pos_weight`, F1 meningkat dari 0,8646 pada threshold default 0,5 menjadi 0,8743 pada threshold optimal 0,9376. Threshold yang lebih tinggi membuat model hanya menandai transaksi yang probabilitasnya sangat tinggi sebagai penipuan, sehingga *precision* naik.

Sebaliknya, menurunkan threshold akan menandai lebih banyak transaksi sebagai penipuan sehingga *recall* meningkat, tetapi *precision* menurun. Pemilihan threshold optimal bergantung pada preferensi risiko: apabila biaya kehilangan penipuan jauh lebih besar daripada biaya pemeriksaan transaksi normal, threshold yang menghasilkan *recall* tinggi lebih disarankan.

8. Kesimpulan

Berdasarkan hasil pemodelan, terdapat beberapa temuan utama.

Dataset *Credit Card Fraud Detection* memiliki distribusi kelas yang sangat tidak seimbang, dengan hanya 0,1727% transaksi berupa penipuan, atau rasio sekitar 578:1.

Ketahanan model *ensemble* terhadap *imbalance* berbeda antar algoritma. Random Forest tetap kompetitif bahkan pada konfigurasi baseline, dengan *recall* 0,8061 dan AUC-PR 0,8584, sedangkan Gradient Boosting baseline jauh lebih lemah dengan *recall* 0,6429 dan AUC-PR 0,4721. Hal ini menunjukkan bahwa pemilihan algoritma *ensemble* sama pentingnya dengan teknik resampling yang digunakan.

Penerapan SMOTE, *undersampling*, dan *class weighting* meningkatkan kemampuan Gradient Boosting secara drastis. SMOTE menaikkan *recall* menjadi 0,8776 dan AUC-PR menjadi 0,8172, sedangkan *class weighting* menghasilkan *recall* 0,8980. Sebaliknya, *undersampling* menghasilkan *recall* tertinggi (0,9184) tetapi dengan *precision* yang sangat rendah (0,0430) karena membuang terlalu banyak data kelas mayoritas.

Tiga implementasi *boosting* modern tampil paling kuat dan konsisten. XGBoost dengan `scale_pos_weight` menjadi model tunggal terbaik secara AUC-PR (0,8847), sedangkan LightGBM unggul pada F1 (0,8649) dan CatBoost mencapai *recall* 0,8571. Ketiganya menangani *imbalance* melalui pembobotan gradien internal sehingga tidak memerlukan data sintetis, dan menghasilkan *false negative* rendah tanpa ledakan *false positive*.

Dari sisi evaluasi, AUC-PR lebih representatif dibanding AUC-ROC untuk dataset yang sangat tidak seimbang, karena AUC-ROC seluruh model tampak tinggi dan sulit dibedakan. Akurasi juga tidak dapat dijadikan metrik utama karena didominasi kelas mayoritas. Terakhir, penyesuaian threshold keputusan dapat meningkatkan F1 model terbaik dari 0,8646 menjadi 0,8743, sehingga pemilihan threshold perlu disesuaikan dengan preferensi risiko.

9. Referensi

[1] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.

[2] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD.

[3] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, 16, 321–357.

[4] He, H., & Garcia, E. A. (2009). *Learning from Imbalanced Data*. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263–1284.

[5] Ke, G., et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS.

[6] Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.

[7] Prokhorenkova, L., et al. (2018). *CatBoost: Unbiased Boosting with Categorical Features*. NeurIPS.